Práctica Final — IA Generativa

FinMind: Asistente Conversacional de Finanzas Personales

3.º Ingeniería de Telecomunicaciones · Curso 2025-2026

1. Caso de uso y rol de la IA generativa

La mayoría de la gente no lleva un control real de sus finanzas. No porque no quiera, sino porque las herramientas actuales son aburridas: te muestran gráficas de barras y categorías de gasto, pero no te dicen nada útil. Ves que has gastado 400€ en «Otros» y no sabes qué hacer con eso.

FinMind es un asistente conversacional que se conecta a tus cuentas bancarias mediante Open Banking (PSD2) y te deja hablar con tus propios datos en lenguaje normal. Puedes preguntarle cosas como «¿en qué he gastado más este mes?», «¿puedo permitirme irme de viaje en agosto?» o «¿cómo puedo ahorrar 200€ al mes sin cambiar demasiado mis hábitos?» y obtener respuestas concretas basadas en lo que tú gastas, no consejos genéricos sacados de un blog.

La idea no es sustituir a un asesor financiero, sino hacer accesible ese tipo de razonamiento a cualquier persona con un smartphone. El LLM actúa como capa de interpretación y comunicación: toma datos estructurados (transacciones, saldos) y conocimiento financiero general, y los convierte en respuestas comprensibles y personalizadas.

2. Arquitectura general

La arquitectura se divide en tres bloques principales. Por un lado, el **frontend**: una app móvil en React Native con una interfaz de chat, que es la única parte que ve el usuario. Por otro, el **backend**, construido con FastAPI, que gestiona la autenticación (OAuth2 + JWT), el enrutamiento y la comunicación con los servicios externos. Y en el centro, el **orquestador**, implementado con LangGraph, que es quien decide qué hacer con cada consulta.

Cuando el usuario manda un mensaje, LangGraph lo analiza y toma una decisión: si la pregunta requiere datos bancarios, lanza una llamada al conector Open Banking (Belvo); si necesita contexto educativo —por ejemplo, que el usuario pregunta qué es un fondo indexado—, recupera fragmentos relevantes del vector store mediante RAG. Lo habitual es que combine ambas cosas a la vez. Todo ese contexto se ensambla en un prompt y se manda al LLM (Claude 3.5 Sonnet), que genera la respuesta final.

Para almacenamiento se usa PostgreSQL a través de Supabase, que guarda el perfil del usuario (metas, presupuesto mensual, preferencias) y el historial de conversaciones. Los embeddings del conocimiento financiero se guardan en Qdrant, que permite filtrar por metadatos como fecha de publicación o categoría temática —útil para no recuperar artículos fiscales desfasados.

3. Técnicas avanzadas implementadas

RAG sobre conocimiento financiero

El sistema indexa una base de conocimiento con artículos de educación financiera, guías de la CNMV y resúmenes de normativa fiscal española (IRPF, deducciones, tramos). Cuando el usuario hace una pregunta conceptual, se recuperan los fragmentos más relevantes y se usan como contexto para que el LLM responda de forma adaptada al perfil concreto del usuario, no con una respuesta genérica.

Se optó por RAG en lugar de fine-tuning por una razón práctica: la normativa fiscal cambia todos los años con los Presupuestos Generales del Estado. Con RAG solo hay que actualizar los documentos indexados; con fine-tuning habría que reentrenar el modelo cada vez que cambia un tramo del IRPF, lo cual no es viable ni en tiempo ni en coste. Además, el fine-tuning no resuelve el problema de alucinaciones sobre datos numéricos personales, que es el riesgo más serio aquí.

Agente con tool-use

Para que el asistente pueda responder preguntas como «¿cuánto he gastado en supermercados este mes?», el LLM necesita acceder a datos en tiempo real. Esto se implementa mediante tool-use: el modelo puede invocar funciones como *get_transactions()* para consultar el historial bancario, *calculate_budget()* para proyecciones de ahorro, o *get_inflation_rate()* para contextualizar si un gasto es mucho o poco. El modelo decide cuándo llamar a cada

herramienta dependiendo de la pregunta, sin que haya lógica hardcodeada para cada tipo de consulta.

Memoria conversacional

LangGraph mantiene el historial de la conversación por sesión y un perfil persistente del usuario con sus metas y presupuesto declarado. Esto permite que el asistente recuerde contexto entre sesiones: si el mes anterior el usuario dijo que quería ahorrar para un viaje, el sistema lo tiene en cuenta al responder preguntas sobre gastos actuales. El perfil se incluye de forma comprimida en el system prompt para no superar la ventana de contexto.

4. Tecnologías empleadas

El LLM principal es **Claude 3.5 Sonnet** de Anthropic, elegido por su buen rendimiento en razonamiento numérico, ventana de contexto larga (200k tokens) y soporte nativo de tool-use. Para los embeddings se usa **text-embedding-3-small** de OpenAI, que tiene un coste bajo y es suficiente para este dominio; una opción más potente como text-embedding-3-large no justifica el sobrecoste para re-indexar documentos frecuentemente.

Como conector de Open Banking se usa **Belvo** en lugar de Plaid, que es la opción más conocida. El motivo es sencillo: Plaid cubre muy bien el mercado americano pero su soporte para bancos españoles (Santander, BBVA, CaixaBank, ING) es bastante limitado. Belvo está certificado bajo PSD2 para el mercado europeo y permite la agregación sin scraping, que está expresamente prohibido por la directiva.

El resto del stack es bastante estándar: **FastAPI** para el backend por su rendimiento y tipado estático, **React Native** para la app móvil (una sola base de código para iOS y Android), **Qdrant** como vector store por su capacidad de filtrado por metadatos, y **LangSmith** para observabilidad de las llamadas al LLM. Todo el stack se puede desplegar sobre Supabase + Railway sin necesidad de infraestructura propia, lo que reduce mucho la complejidad operativa para una primera versión.

5. Multimodalidad

El sistema trabaja con cuatro tipos de datos. El más básico son las **transacciones bancarias** en formato JSON (importe, comercio, categoría, fecha), que llegan del conector Belvo. También se procesan **documentos en PDF**: tanto los artículos financieros de la base de conocimiento como documentos que el propio usuario puede subir, como un contrato de hipoteca o una póliza de seguro, para que el asistente los interprete.

La parte más interesante desde el punto de vista multimodal es el procesamiento de **imágenes**. Claude 3.5 acepta imágenes de forma nativa, así que el usuario puede hacer una foto a un ticket o una factura y el sistema extrae el importe y lo categoriza automáticamente. Esto elimina la fricción principal de las apps de finanzas: la gente abandona porque no quiere introducir gastos manualmente. Con la foto basta. Por último, los datos de evolución de gasto (series temporales numéricas) se pasan al modelo como resumen estadístico o como gráfica adjunta para contextualizar tendencias.

6. Evaluación

Para evaluar el sistema se usará el framework **RAGAS**, que permite medir de forma automatizada las métricas clave de un pipeline RAG. Las más relevantes para este caso son *faithfulness* (que el modelo no invente cifras que no están en los datos del usuario), *context precision* (que los fragmentos recuperados sean realmente útiles) y *answer relevance* (que la respuesta conteste lo que se preguntó). Los umbrales objetivo son 0.92 para faithfulness, 0.80 para context precision y 0.85 para answer relevance.

El de faithfulness es el más crítico: si el sistema le dice al usuario que gastó 500€ en ropa cuando en realidad gastó 50€ y el usuario toma una decisión financiera basándose en eso, el daño es difícil de reparar. También se medirá la precisión del OCR en tickets fotográficos, con un umbral del 95%, ya que errores de extracción ahí se propagan directamente al historial de gastos.

Para construir el dataset de evaluación se recopilarán 150 preguntas financieras típicas con sus respuestas correctas, validadas por un asesor financiero. Las métricas RAGAS se calcularán automáticamente en cada despliegue como parte del pipeline CI/CD. A nivel de producto, se medirá la retención a 30 días (objetivo ≥40%) y la satisfacción del usuario con una pregunta simple al final de cada conversación (escala 1-5, objetivo ≥4.0).

7. Seguridad y privacidad

Datos bancarios y cumplimiento normativo

Los datos financieros son especialmente sensibles y están regulados bajo el RGPD y la directiva PSD2. El sistema no almacena nunca las credenciales bancarias del usuario: Belvo trabaja con tokens OAuth de corta duración que solo permiten lectura de movimientos. Las transacciones se guardan en PostgreSQL con cifrado a nivel de columna usando pgcrypto, y cada usuario tiene su propia clave derivada de su contraseña maestra, que nunca sale del dispositivo. El usuario puede exportar o borrar todos sus datos en cualquier momento, en cumplimiento del derecho de supresión del RGPD.

Riesgos específicos del pipeline LLM

El mayor riesgo técnico es la **alucinación de cifras**: el modelo podría generar importes que no corresponden a los datos reales. Para mitigarlo, todos los valores numéricos en la respuesta se validan contra los datos originales antes de mostrarse; si hay discrepancia, la UI muestra los datos reales con una advertencia visible. El system prompt también obliga al modelo a citar siempre la fuente de cada dato concreto.

El **prompt injection** es otro vector a considerar: si un tercero pudiera insertar instrucciones en el nombre de un comercio o en la descripción de una transferencia, podría manipular el comportamiento del modelo. Se mitiga separando claramente en el prompt los datos externos (transacciones) del contexto del sistema, con delimitadores explícitos, y el LLM recibe instrucciones de ignorar cualquier directiva que aparezca en los bloques de datos.

En cuanto al **control de acceso**, la autenticación en la app usa biometría (Face ID o huella dactilar) como segundo factor, sesiones JWT de 15 minutos con refresh token, y cada acceso queda auditado con IP, dispositivo y timestamp. Por último, el sistema deja claro en todo momento que no es un asesor financiero regulado: las recomendaciones sobre inversión incluyen siempre la advertencia correspondiente y se limitan a conceptos generales, sin sugerir productos específicos.

En conjunto, FinMind es un caso de uso donde los LLMs aportan valor real: no como sustituto de un profesional, sino como capa de inteligencia sobre datos que el usuario ya tiene pero no sabe cómo leer.